Consumer Decision-Making Analysis in R

Reproducible, Collaborative, and AI-Aware Workflows

THE DATAflow COMPANY

Who we are

Jannis
Kreienkamp

Statistics
Software

Maximilian
Agostini

Data OPS
Infrastructure

Why this workshop?

Overview

Structure of the workshop

- ❑ Day 1: The scaffold
- ❑ Day 2: R and data analysis
- ❑ Day 3: Producing a report + Supercharge

The three-day roadmap

Day 1: Scaffold

- R
- Rstudio
- Git/GitHub basics
- Folders
- Data protection
- Quarto

Day 2: Analyse

- Base R
- Import
- Clean
- Transform
- Visualise
- Describe
- Model
- Moderate / mediate
- Collaborate

Day 3: Publish

- Export tables and figures
- Write methods
- Write results
- Static/ Annotated/ interactive report
- AI-use statement
- Reproducibility check

THE DATAflow COMPANY

Day 1

The Repository

What, where, why?

A quick vocabulary check

```
fork         commit        push / pull       branch / merge       stage
repository   RStudio       .Rproj            renv                 snapshot
Quarto       render        .qmd              GitHub Pages         Overleaf
```

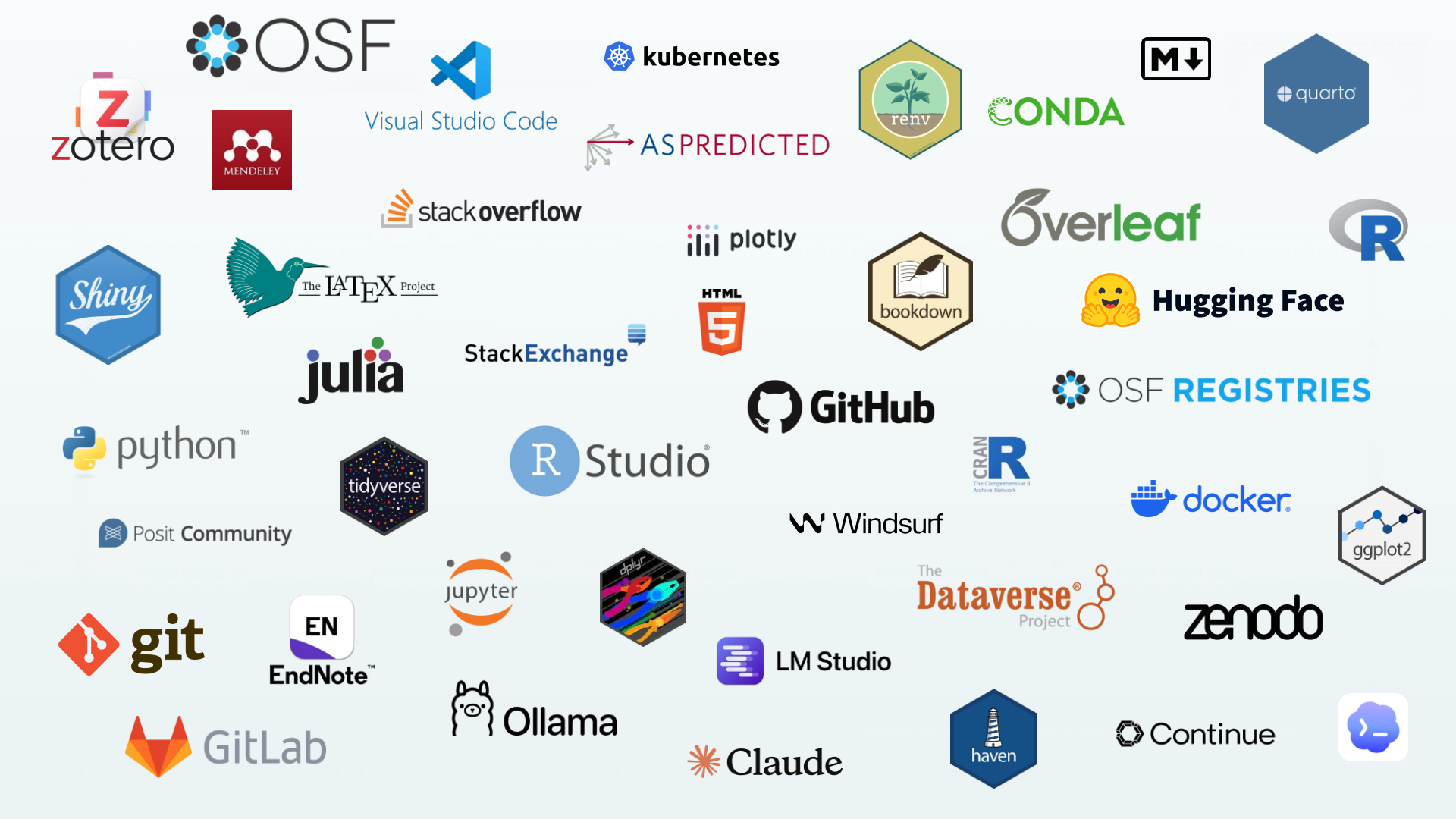
Don't know them yet? Perfect. After today, we hope you do.

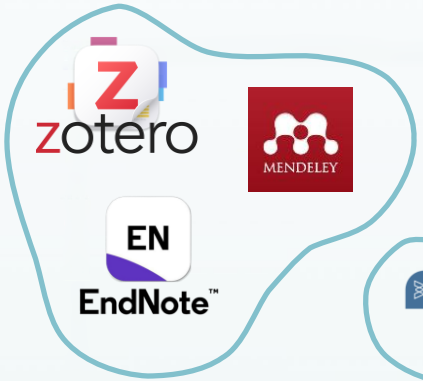
Setting expectations — not today

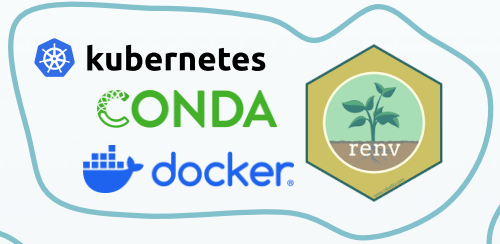
```
Deep R syntax        → Day 2
VS Code              → Day 3
Real data            → Day 2/3
Responsible AI use   → Day 3
```

Today's single job: install the tools and understand how they fit together.

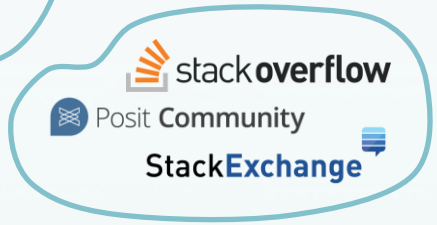
Good Scientist Recipe


OSF
kubernetes
zotero
Visual Studio Code
MENDELEY
ASPREDICTED
renv
CONDA
quarto
stackoverflow
plotly
Overleaf
Shiny
The LaTeX Project
HTML
bookdown
Hugging Face
julia
StackExchange
OSF REGISTRIES
GitHub
python
tidyverse
RStudio
CRAN
docker
Posit Community
Windsurf
ggplot2
jupyter
dplyr
The Dataverse Project
zenodo
git
EN
EndNote
LM Studio
Ollama
GitLab
Claude
haven
Continue


zotero
MENDELEY
EN
EndNote™


kubernetes
CONDA
docker
renv


M↓
The LaTeX Project
quarto
bookdown
Overleaf

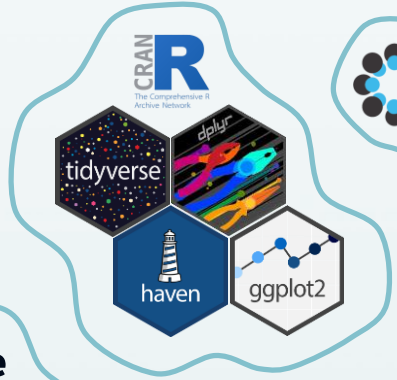

stackoverflow
Posit Community
StackExchange

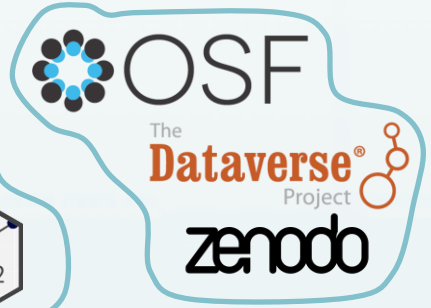

plotly
HTML
Shiny


AS PREDICTED
OSF REGISTRIES

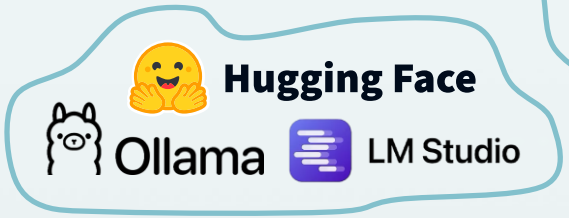

python™
julia
R


RStudio
Visual Studio Code
jupyter
Windsurf

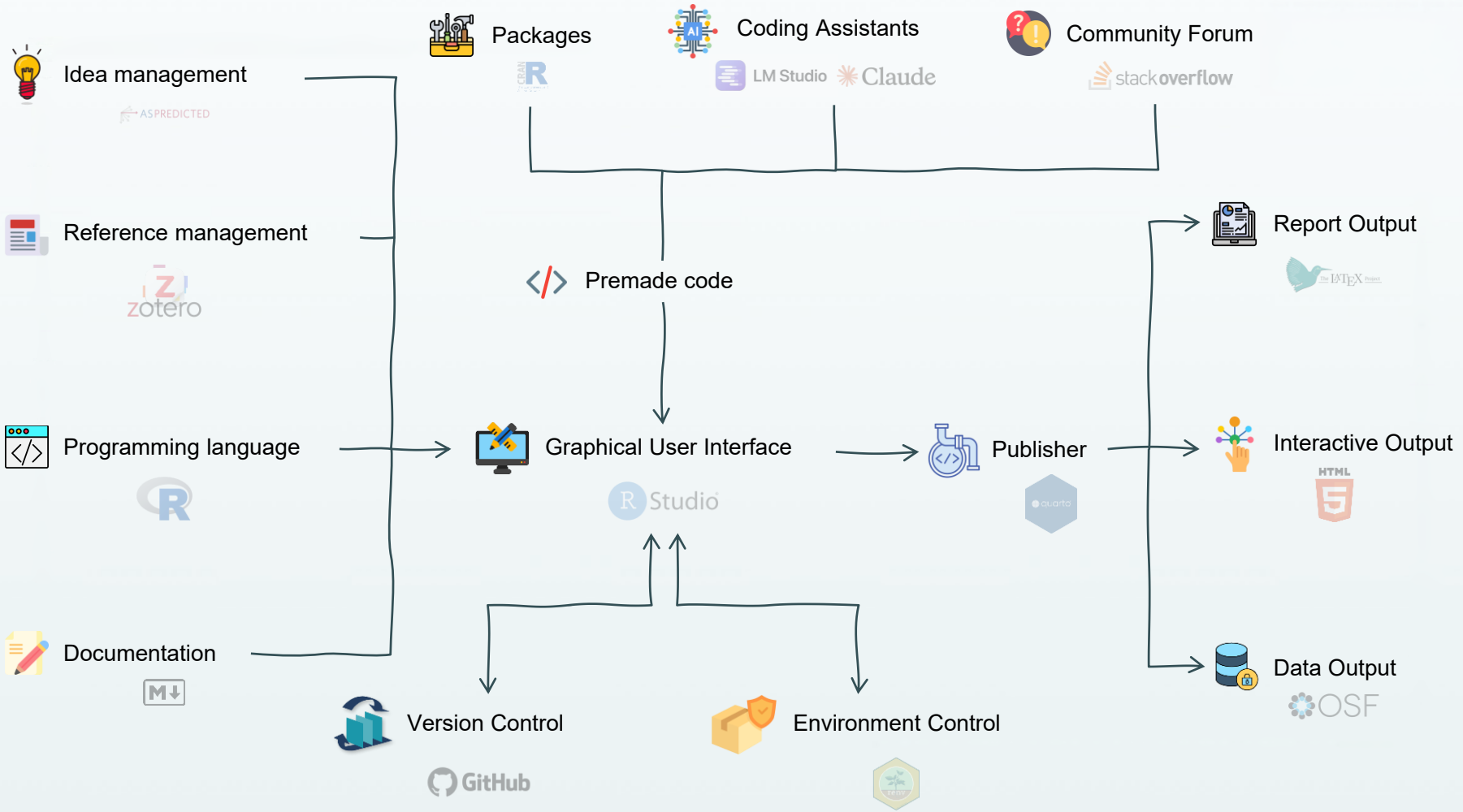

CRAN R
The Comprehensive R Archive Network
tidyverse
dplyr
haven
ggplot2

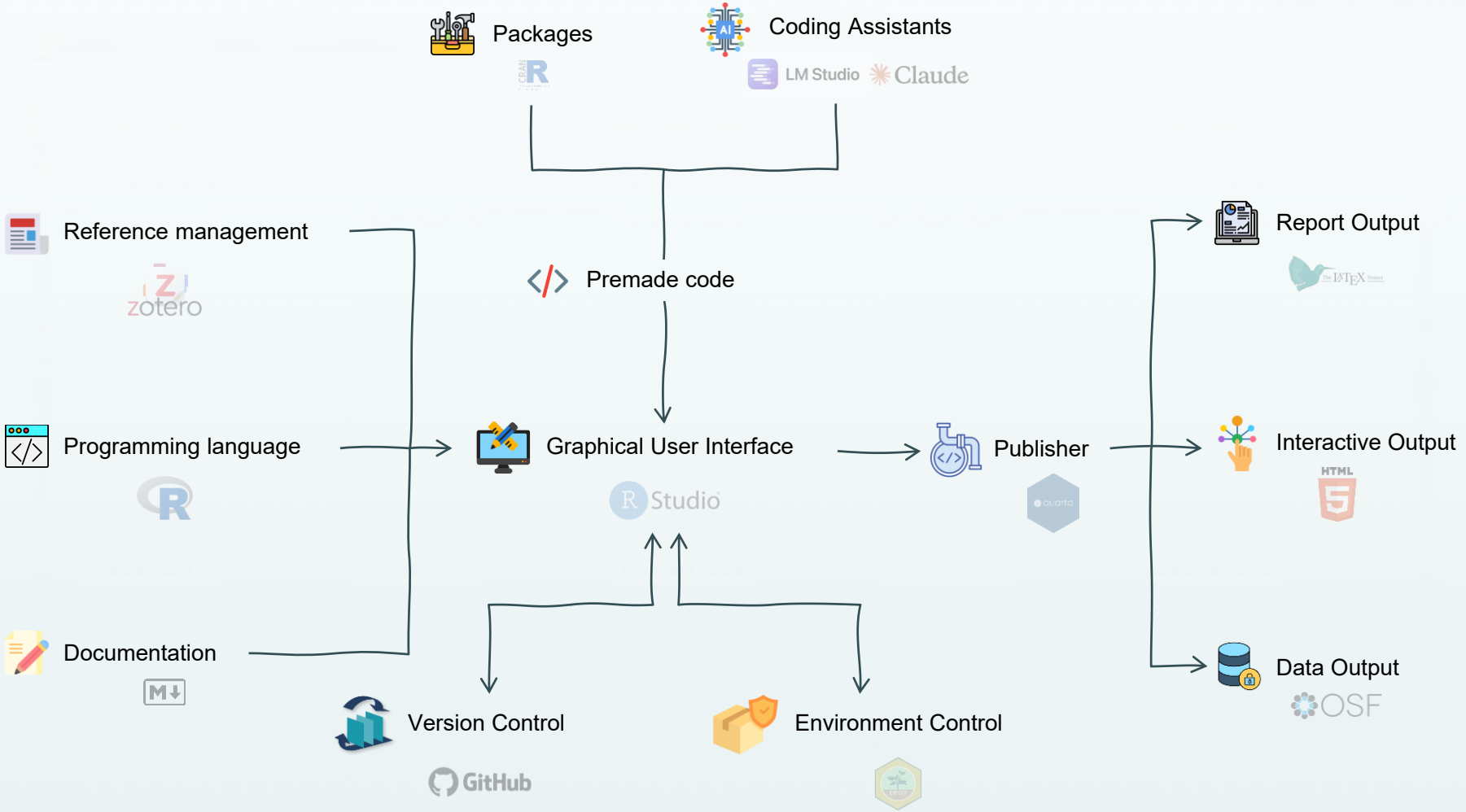

OSF
The Dataverse® Project
zenodo


git
GitHub
GitLab


Hugging Face
Ollama
LM Studio


Claude
Continue


Packages
Coding Assistants
LM Studio
Claude
Community Forum
stackoverflow
Idea management
ASPREDICTED
Reference management
zotero
Premade code
Report Output
Programming language
Graphical User Interface
Studio
Publisher
quarto
Interactive Output
HTML
Documentation
Version Control
GitHub
Environment Control
Data Output
OSF


Packages
Coding Assistants
LM Studio
Claude
Reference management
zotero
Premade code
Report Output
Programming language
Graphical User Interface
Studio
Publisher
Interactive Output
HTML
Documentation
Version Control
GitHub
Environment Control
Data Output
OSF

Every tool today is free and open source (FOSS)

R & friends	free; huge community; you keep your analysis (vs SPSS: €€€, closed)
Markdown	plain text; machine-readable; no vendor can lock or change it (vs Word)
Git / GitHub	open standard; the full history is yours to keep and to move
Zotero	free, open reference manager; plugs into Word, Quarto, Overleaf
Quarto	free, open publishing; one source → website, PDF, Word

Open + free + standard = reproducible, shareable, and future-proof.

No licence to expire. No format you can't open in ten years.

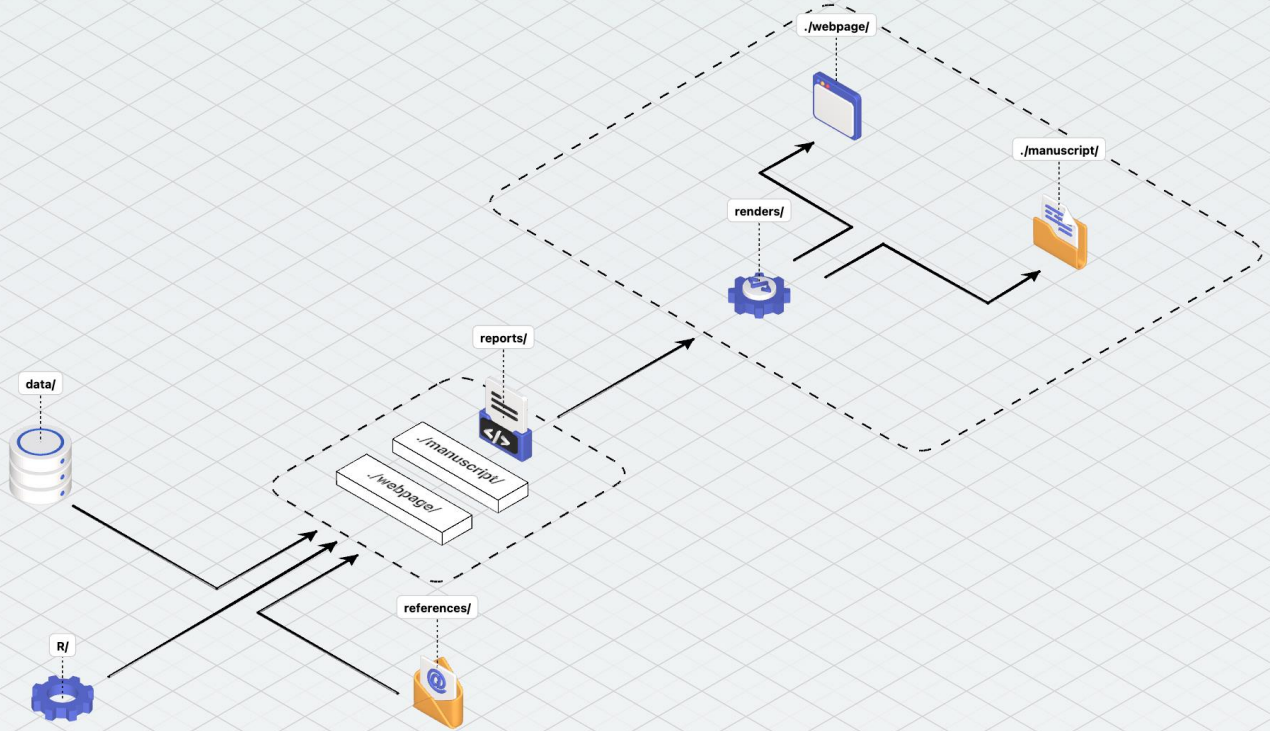
THE DATAflow COMPANY

the-science-repository Public

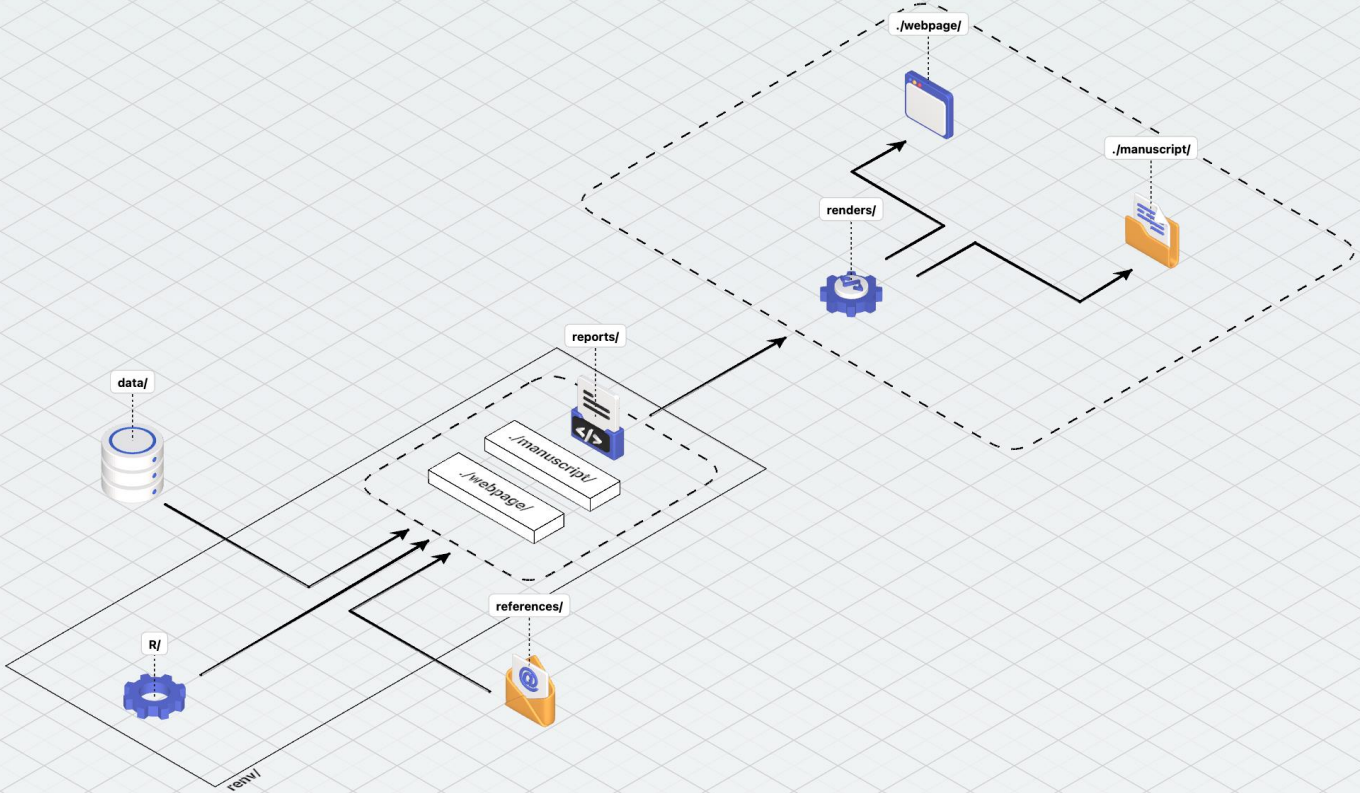
A template for reproducible science in R: one repo for your data, analysis, interactive render, and manuscript.

R ☆ 5

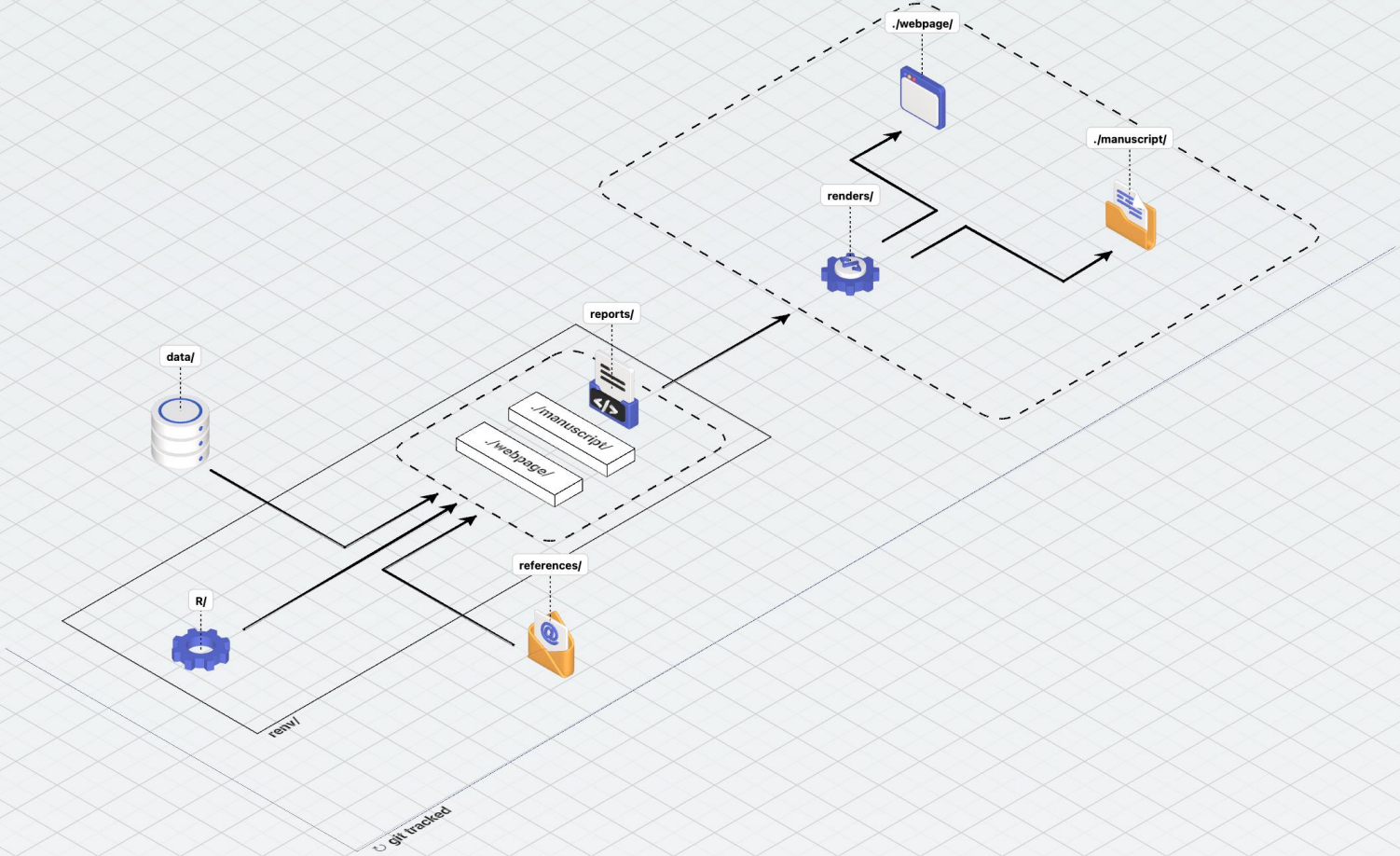
the-science-repository/



the-science-repository/



the-science-repository/



A quick tour of the folders (the kitchen)

```
the-science-repository/
│
├── data/                    INGREDIENTS
│
├── R/                       APPLIANCES & TECHNIQUES
│
├── reports/                 THE RECIPES
│
├── renders/                 THE PLATED DISHES
│
└── references/              THE NUTRITION FACTS
```

A quick tour of the folders (the kitchen)

```
the-science-repository/
├── README.md                 Instructions in every folder
│
├── data/                     INGREDIENTS
│   ├── README.md
│   ├── mock/                 ←  display/sample ingredients (safe to share, committed)
│   ├── raw/                  ←  ingredients as delivered (private, gitignored)
│   └── processed/            ←  prepped ingredients (private, reproducible)
│
├── R/                        APPLIANCES & TECHNIQUES
│   ├── README.md
│   └── functions/            ←  reusable base techniques (load, model, plot)
│
├── reports/                  THE RECIPES
│   ├── webpage/              ←  recipe for the website dish
│   └── manuscript/           ←  recipe for the paper dish
│
├── renders/                  THE PLATED DISHES
│   ├── webpage/              ←  the served website (100% generated)
│   └── manuscript/           ←  the served paper (mixed: hand- + machine-plated)
│       ├── generated/        ←  machine-plated parts (figures, tables, methods/results)
│       └── sections/         ←  hand-plated parts (abstract, intro, discussion)
│
└── references/               THE NUTRITION FACTS (inspiration/ shared references.bib)
```

A quick tour of the folders (the kitchen)

```
the-science-repository/
├── README.md                 Instructions in every folder
│
├── data/                     INGREDIENTS
│   ├── README.md
│   ├── mock/                 ←  display/sample ingredients (safe to share, committed)
│   ├── raw/                  ←  ingredients as delivered (private, gitignored)
│   └── processed/            ←  prepped ingredients (private, reproducible)
│
├── R/                        APPLIANCES & TECHNIQUES
│   ├── README.md
│   └── functions/            ←  reusable base techniques (load, model, plot)
│
├── reports/                  THE RECIPES
│   ├── webpage/              ←  recipe for the website dish
│   └── manuscript/           ←  recipe for the paper dish
│
├── renders/                  THE PLATED DISHES
│   ├── webpage/              ←  the served website (100% generated)
│   └── manuscript/           ←  the served paper (mixed: hand- + machine-plated)
│       ├── generated/        ←  machine-plated parts (figures, tables, methods/results)
│       └── sections/         ←  hand-plated parts (abstract, intro, discussion)
│
└── references/               THE NUTRITION FACTS (inspiration/ shared references.bib)
```

A quick tour of the folders (the kitchen)

```
the-science-repository/
├── README.md                    Instructions in every folder
│
├── data/
│   ├── README.md
│   ├── mock/
│   │   └── README.md
│   ├── raw/
│   │   └── README.md
│   └── processed/
│       └── README.md
│
├── R/
│   ├── README.md
│   └── functions/
│       └── README.md
│
├── reports/
│   ├── webpage/
│   │   └── README.md
│   └── manuscript/
│       └── README.md
└── ...
```

Every project explains itself

THE DATAflow COMPANY

A README tells you:
- where you are in the project
- what lives in the folder
- how to set up and run things

READMEs are written in Markdown

THE DATAflow COMPANY

```
# Heading level 1
## Heading level 2

**bold**, *italic*, ~~strikethrough~~

- bullet point
- another bullet

[My link label](https://example.com)

---

```R
# multiple lines of
# code instructions
```

My `variable name` looks like code

:+1:
```

Heading level 1

Heading level 2

bold, *italic*, ~~strikethrough~~

- bullet point
- another bullet

My link label

```
# multiple lines of
# code instructions
```

My `variable name` looks like code

👍

cheat sheet

READMEs are written in Markdown

```
## Why Markdown instead of Word?

A README written in Markdown is still readable even
before it is rendered.

### Git-friendly

- Plain text → every change is visible
- Works naturally with version control (`git diff`)

### Predictable

> What you write is what everyone sees.

- No hidden formatting
- No accidental style changes
- No "Why did Word move everything again?"

### Future-proof

- Open format → no vendor lock-in
- Readable in any text editor
- Likely to remain accessible for decades
- Readable by programs and AI
```

Why Markdown instead of Word?

A README written in Markdown is still readable even before it is rendered.

Git-friendly

- Plain text → every change is visible
- Works naturally with version control (`git diff`)

Predictable

> What you write is what everyone sees.

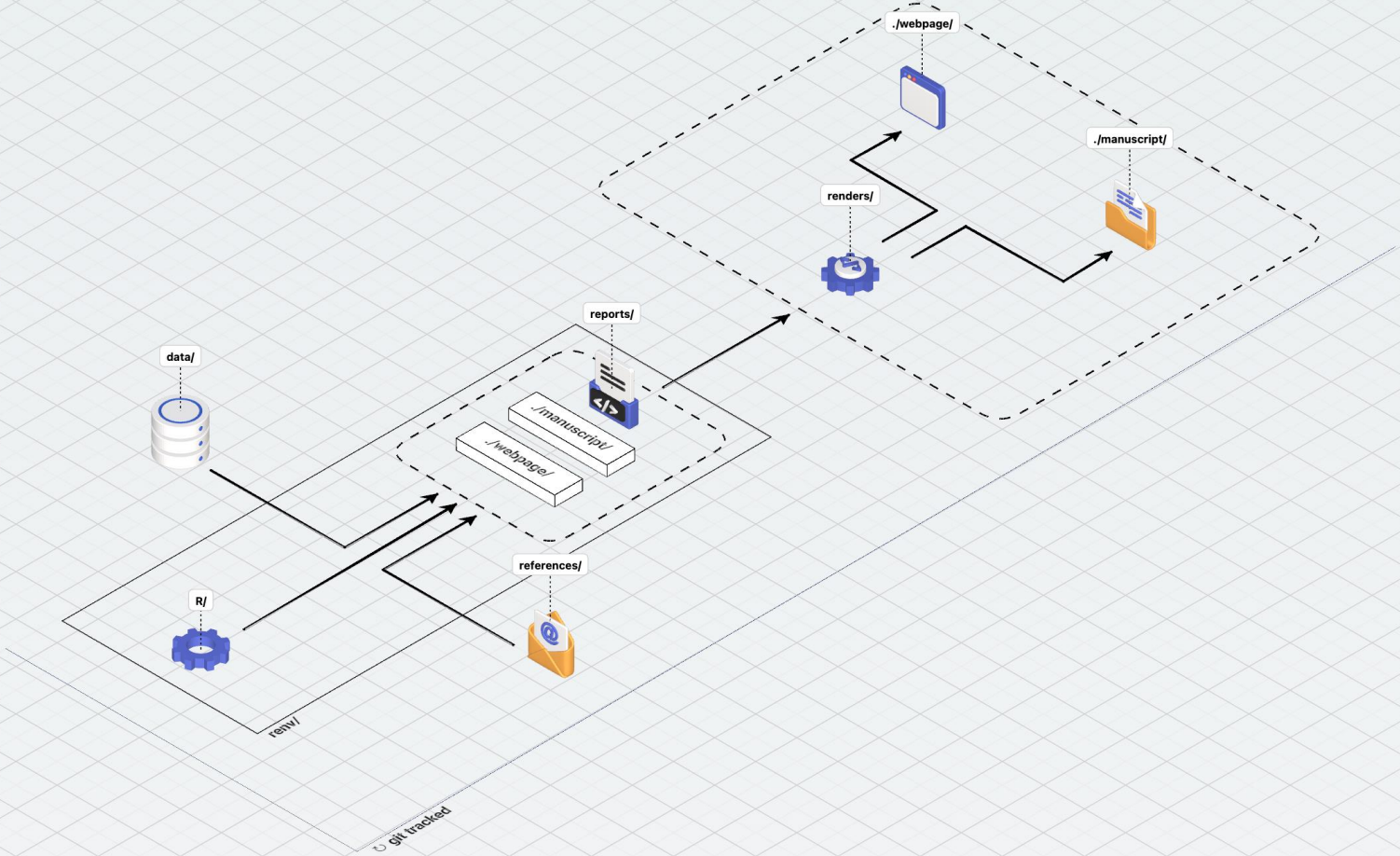
- No hidden formatting
- No accidental style changes
- No “Why did Word move everything again?”

Future-proof

- Open format → no vendor lock-in
- Readable in any text editor
- Likely to remain accessible for decades
- Readable by programs and AI

cheat sheet

the-science-repository/



Version control: the safety net

git

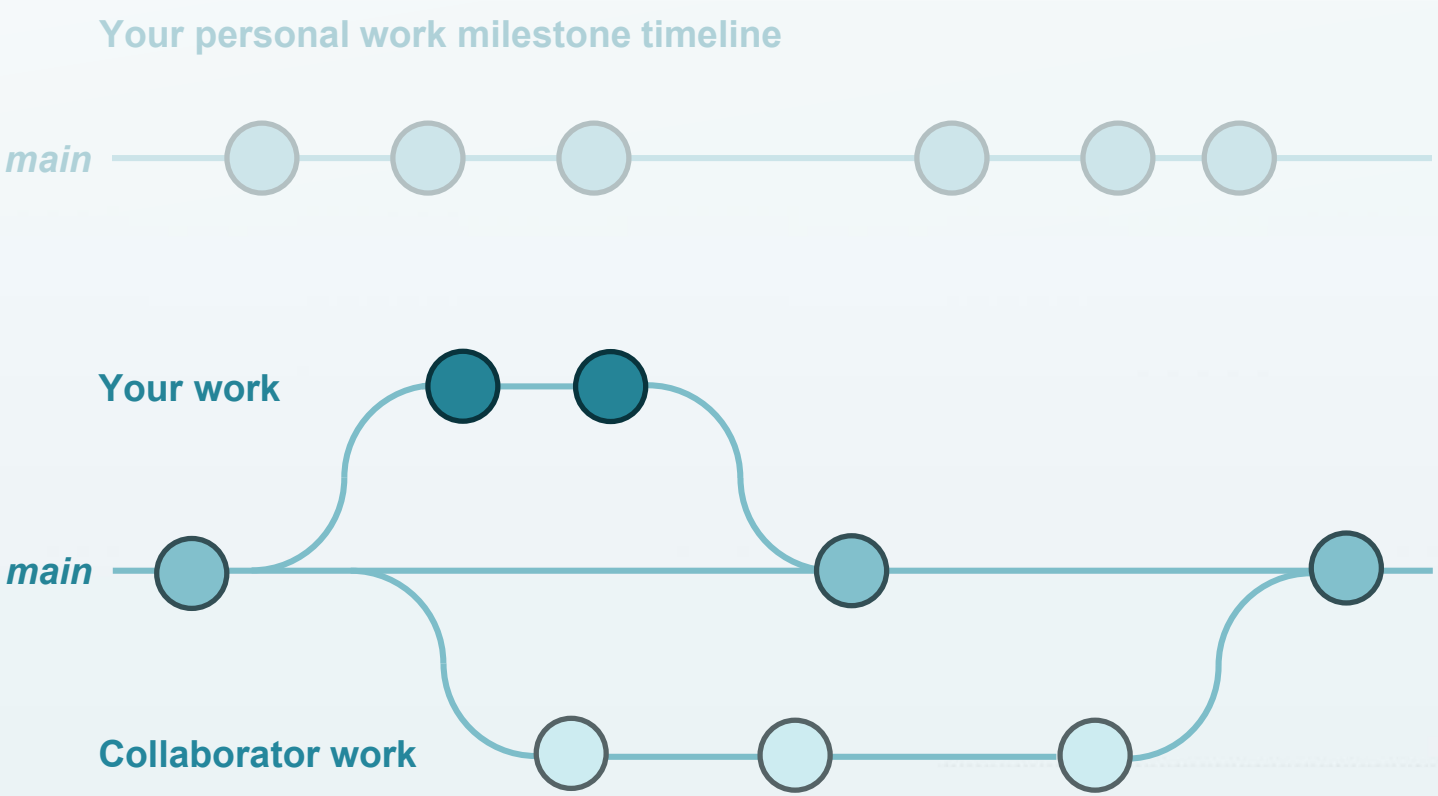
GitHub

Why should I care

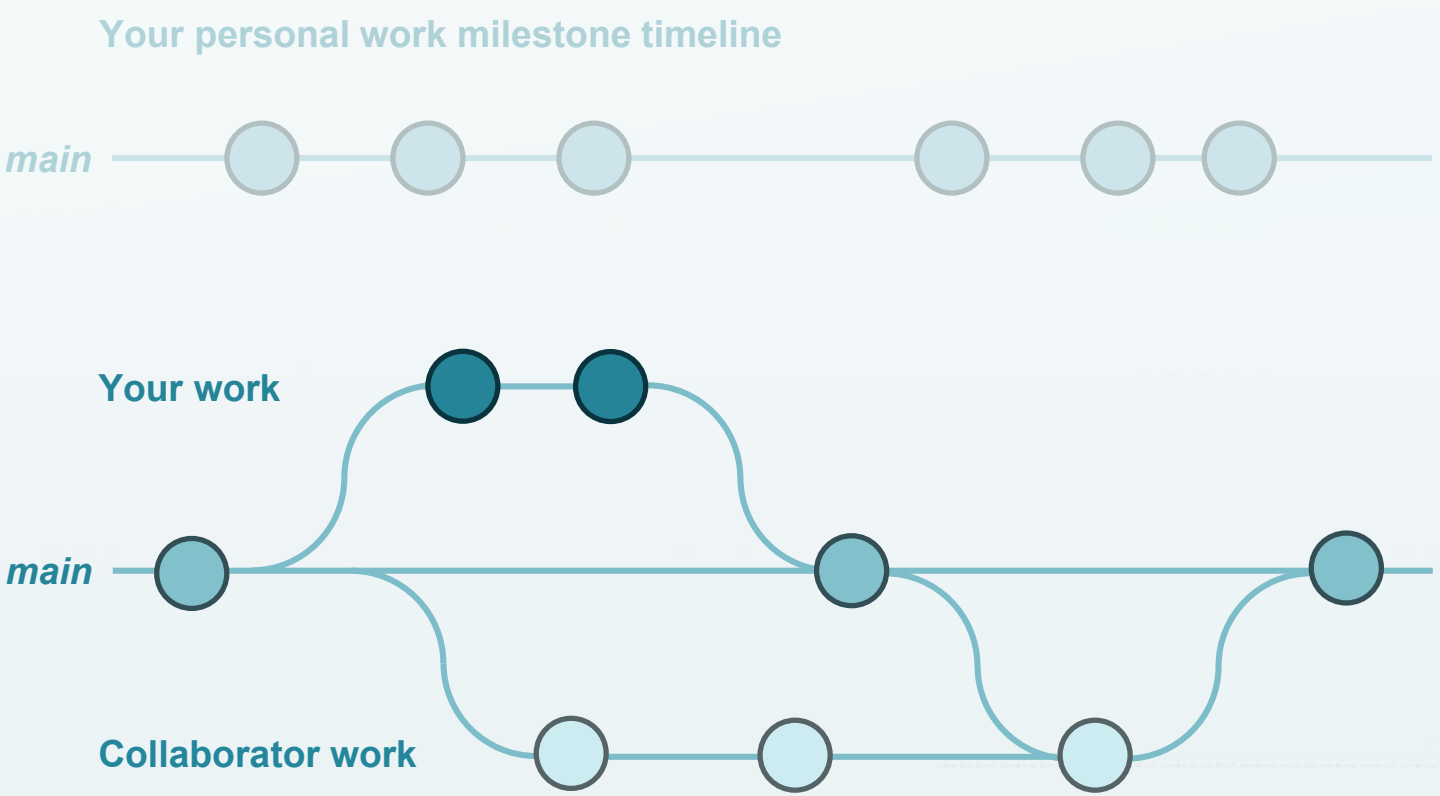
THE DATAflow COMPANY

Your personal work milestone timeline

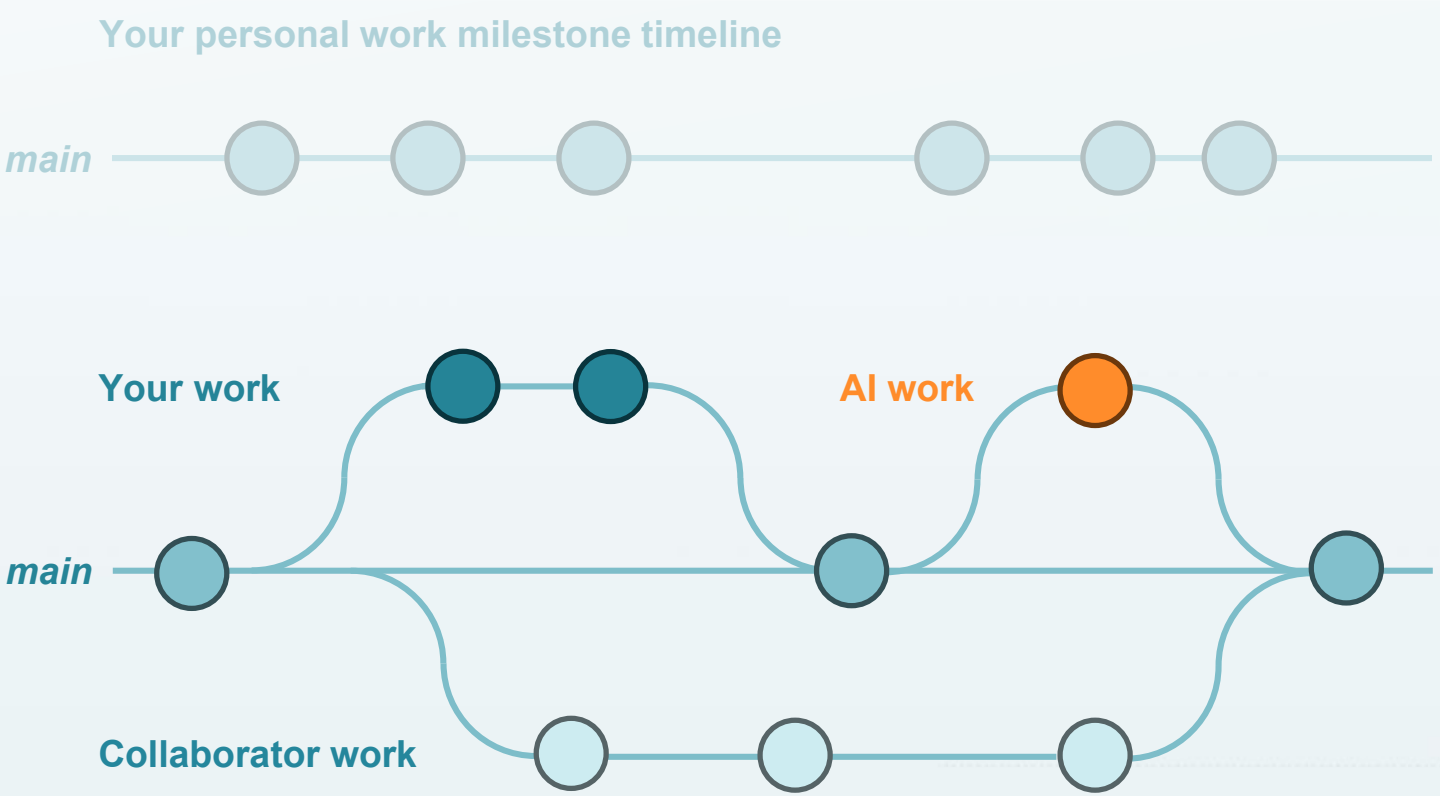
Why should I care



Why should I care



Why should I care



The words you'll hear all day

Fork	Make your own copy of someone else's repo on GitHub
Clone	Download a repo to your computer
Stage	Pick which changes go into the next commit
Commit	Save a labelled snapshot of the staged changes
Push	Send your commits up to GitHub
Pull	Bring GitHub's changes down to your computer
Branch	A separate line of work
Merge	Combine work from different branches

Errors are part of the process

Errors are normal. Everyone in this room will see red text today.
An error message is a clue, not a failure.

When you hit one:

1. Read it slowly — the answer is usually right there (a path, a name, a missing thing)
2. Recall the last thing you did — the error points back to it
3. Copy the exact message into a search engine — someone has hit it before
4. Ask a neighbour, or ask us

Step 1 — get an account

1. Go to github.com
2. Sign up (use an email you'll keep — ideally your university email)
3. Verify your email
4. Pick a username you're happy to share publicly

CHECK

Can you log in and see your (empty) GitHub dashboard?

WATCH OUT

- Verification email can be slow or land in spam — check both.
- Some corporate/university mail servers block the verification link (check the spam)
- The username becomes part of your public website URL later — choose something you won't regret printing on a paper.

Step 2 — make it yours

1. Open: github.com/thedataflowcompany/the-science-repository
2. Click "Fork" (top right)
3. Keep the name, click "Create fork"
4. You now have: github.com/<your-awesome-username>/the-science-repository

WATCH OUT

- If you've forked this repo before, GitHub won't fork it twice — just open your existing copy.
- Don't accidentally fork into an organisation; fork into your own account.

LATER — STARTING YOUR OWN PROJECT (NOT TODAY)

Forking keeps a visible link to this template — ideal for the workshop.
For your own real project, start a clean, independent repo instead:

1. Code → Download ZIP (a snapshot with no fork link)
2. Create a NEW repo on GitHub — public, or private if your data or project demands it
3. Copy the files in, commit, and push

Today we all stay on the public fork.

Step 3 — publish your analysis website

Preview | Code | Blame | 87 lines (66 loc) · 5.02 KB | Raw

```
# The Science Repository

A template for a modern, reproducible science project in R. One git repository
holds the **data**, the **analysis engine**, and two polished outputs built from
it: a **website** that shows the whole analysis transparently, and a **journal
manuscript**. Fork it, rename it, and fill it with your own work.

> 📊 **[Open the full analysis website →](https://thedataflowcompany.github.io/the-science-repository/renders/webpage/)**
> Every cleaning step, figure, table, and model, rendered from the code in this repo.

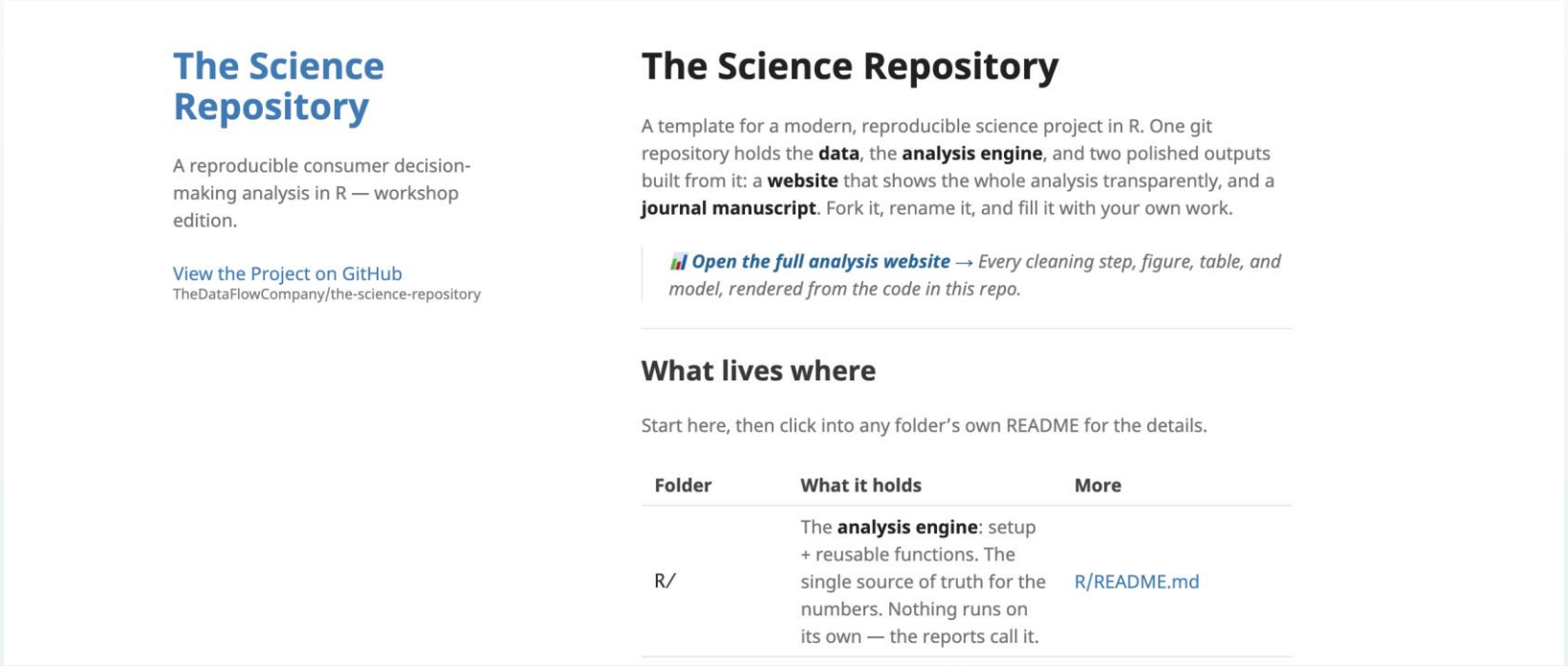
---

## What lives where

Start here, then click into any folder's own README for the details.

| Folder | What it holds | More |
| --- | --- | --- |
| [`R/`](R/) | The **analysis engine**: setup + reusable functions. The single source of truth for the numbers. Nothing runs on its own
```

Step 3 — publish your analysis website



The Science Repository

A reproducible consumer decision-making analysis in R — workshop edition.

View the Project on GitHub
TheDataFlowCompany/the-science-repository

The Science Repository

A template for a modern, reproducible science project in R. One git repository holds the **data**, the **analysis engine**, and two polished outputs built from it: a **website** that shows the whole analysis transparently, and a **journal manuscript**. Fork it, rename it, and fill it with your own work.

> 📊 ***Open the full analysis website*** → *Every cleaning step, figure, table, and model, rendered from the code in this repo.*

What lives where

Start here, then click into any folder's own README for the details.

Folder	What it holds	More
R/	The **analysis engine**: setup + reusable functions. The single source of truth for the numbers. Nothing runs on its own — the reports call it.	R/README.md

Step 3 — publish your analysis website

In YOUR fork on GitHub:
1. Settings → Pages
2. Source: "Deploy from a branch"
3. Branch: main · Folder: / (root) → Save
4. Wait ~1 minute, then open:
   https://<your-username>.github.io/the-science-repository
5. While waiting, apply for GitHub Education

WATCH OUT

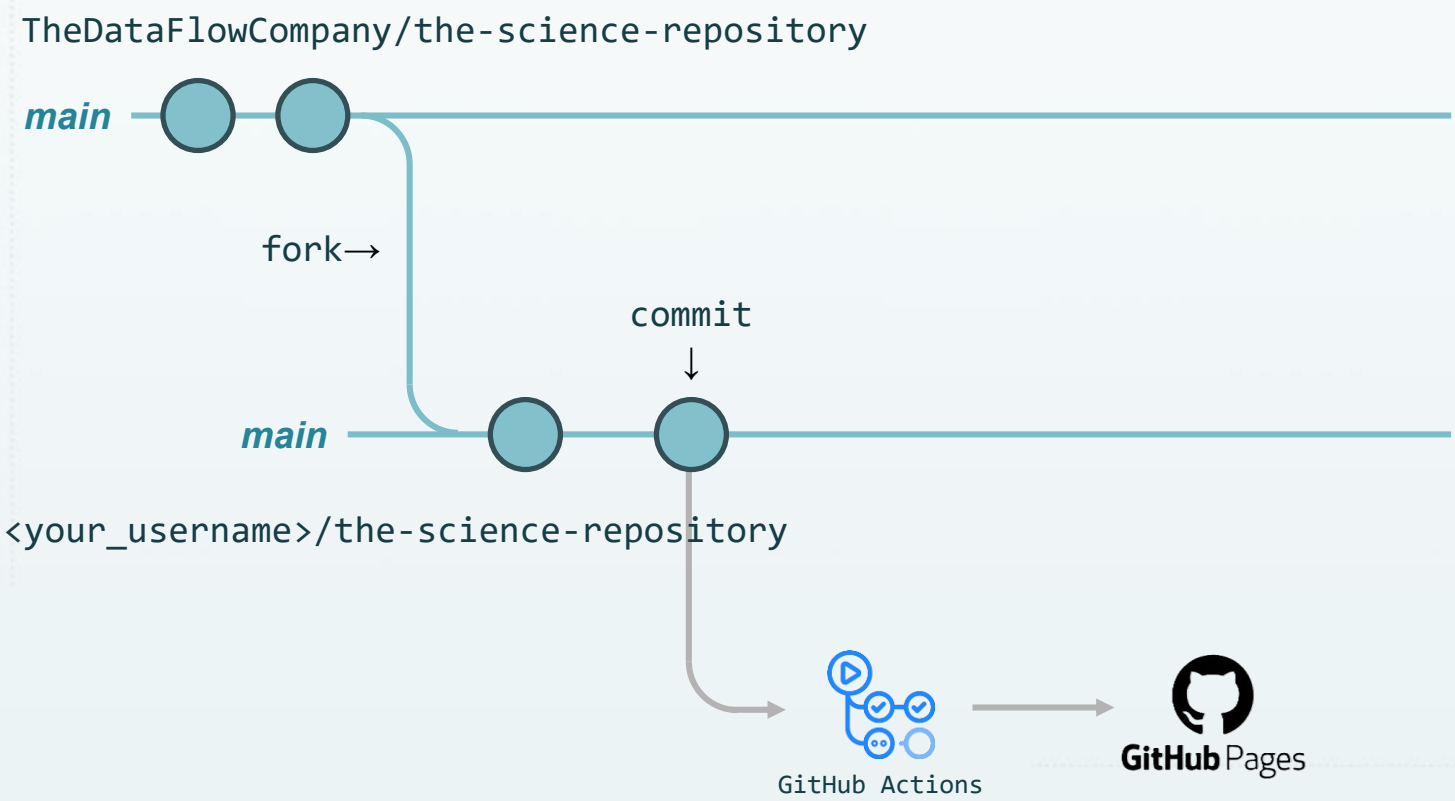
- GitHub Pages on a FREE account needs a PUBLIC repo. Your fork of a public template is public by default, so you're fine. A private repo needs GitHub Pro / Education for Pages.
- First build takes 1–2 minutes. A 404 usually means the build isn't done yet, or the URL is wrong — it must end with /renders/webpage/ .

Step 4 — edit the README link and commit

The README points to the project's website. Make it point to YOURS.

1. Open README.md on GitHub → click the pencil (edit)
2. Find the website link near the top
3. Replace the URL with your own github.io address (from the previous step)
4. Scroll down → "Commit changes"
   Message: "Point website link to my fork" or anything else you like

What just happened, visually



The words you'll hear all day

~~Fork~~	~~Make your own copy of someone else's repo on GitHub~~
Clone	Download a repo to your computer
Stage	Pick which changes go into the next commit
~~Commit~~	~~Save a labelled snapshot of the staged changes~~
Push	Send your commits up to GitHub
Pull	Bring GitHub's changes down to your computer
Branch	A separate line of work
Merge	Combine work from different branches

Data protection checkpoint .gitignore

THE DATAflow COMPANY

Usually safe to commit:

- scripts / code
- README & docs
- mock (synthetic) data
- rendered outputs

Usually NOT safe to commit:

- real / identifiable participant data
- passwords, tokens, API keys
- ethics docs with sensitive details
- anything confidential

Rule of thumb:

Scripts can be shared. Raw data usually cannot.
Outputs depend on whether anyone can be identified.

Real data can't be committed by accident

```
data/mock/        committed     (synthetic, safe — everyone uses this today)
data/raw/         gitignored    (your real data — never leaves your machine)
data/processed/   gitignored    (cleaned data — also stays local)

.gitignore also blocks common data file types everywhere except mock/.
```

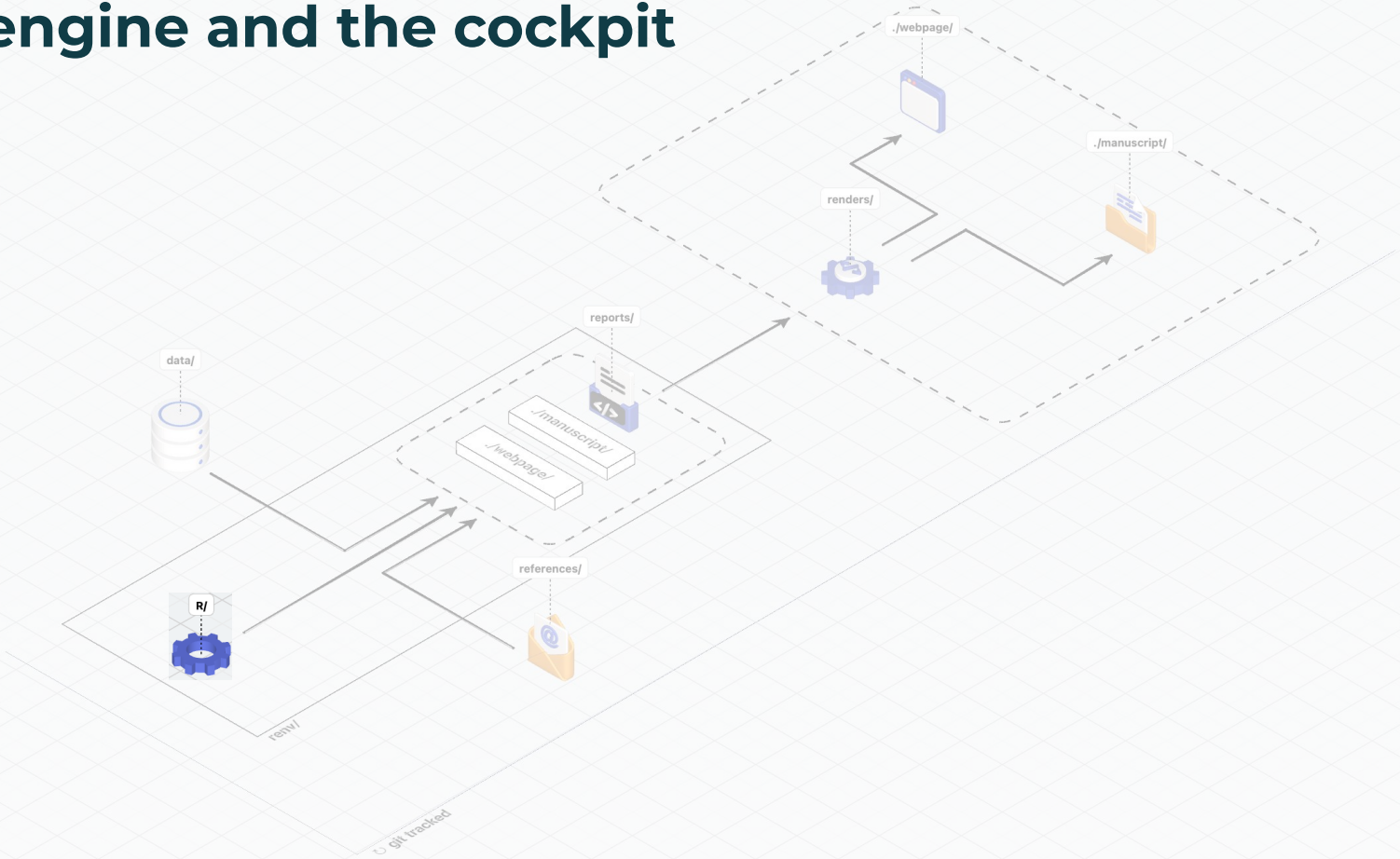
BREAK

Coffee break — 10:30–11:00

Back at 11:00.

✓ So far: GitHub account · your fork · website live on Pages · first commit
→ Next: install R & RStudio, then clone your fork to your own laptop

The engine and the cockpit



The engine and the cockpit

```
R        = the programming language that does the work → the stove
RStudio  = the environment you work in: write code, see data,
           run scripts, view plots, manage the project (the cockpit)

You can run R without RStudio – but RStudio makes everything visible.
```

Step 5 — install the software

Go to: https://posit.co/download/rstudio-desktop (has links to both)

1. Install R: cran.r-project.org (pick your OS, latest version)
2. Install RStudio: posit.co/download/rstudio-desktop (free Desktop)
3. Open RStudio — it finds R automatically

CHECK

Does RStudio open without errors?
In the Console, type 1 + 1 and press Enter → you should see [1] 2

WATCH OUT

- Install R FIRST, then RStudio (RStudio needs R to exist).
- University laptops often need admin rights
- Mac: pick the build for your chip (Apple Silicon vs Intel).
- Antivirus on Windows can quarantine the installer — allow it.

Step 6 — clone your fork (1 / 2)

Cloning = downloading YOUR fork (with its full history) to your machine.

In RStudio:
  File → New Project → Version Control → Git
  Repository URL: https://github.com/<your-username>/the-science-repository → Create Project

Two ways to handle the GitHub login — pick one:
  A. GitHub Desktop (easiest today)
     Install desktop.github.com → sign in (in the browser) → File → Clone
     No Personal Access Token to set up.
  B. Personal Access Token / PAT (the sustainable, long-term setup)
     In the R Console:
        usethis::create_github_token()   # opens GitHub to create the token
        gitcreds::gitcreds_set()   # paste it once; RStudio remembers it
     Alternative:
        Go to Github/Settings/Developer Settings/Personal Access Tokens/PAT

Heads up: this is the fiddliest step of the whole day. That's expected —
we will get everyone through it. Take a breath; ask for help.

Step 6 — clone your fork (2 / 2)

THE DATAflow COMPANY

WATCH OUT

- If the "Git" option is greyed out, Git isn't installed → install from git-scm.com, restart RStudio.
- FIRST TIME ONLY — tell Git who you are (same email as GitHub):

```
git config --global user.name  "Your Name"
git config --global user.email "you@university.edu"
```

- Pushing over HTTPS no longer accepts your password. When asked, paste a Personal Access Token (option B), or use GitHub Desktop (option A) which signs you in through the browser and handles credentials for you.

Always open via the .Rproj

the-science-repository.Rproj  x2

Opening the project (not single files) tells R:
- where the project root is
- where data, scripts, and outputs live

→ file paths work the same on everyone's computer

How RStudio Projects get created

```
File → New Project → …
  1. New Directory       a brand-new empty project folder
  2. Existing Directory  wrap a project around a folder you already have
  3. Version Control     clone from GitHub (what we just did)

Every option produces a  .Rproj  file = the project's anchor.
```

BREAK

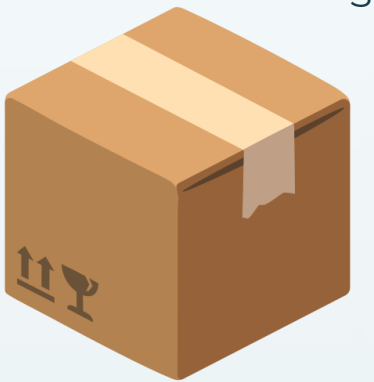
Lunch — 12:30–13:30

Back at 13:30.

✓ So far: R & RStudio installed · fork cloned · project open via the .Rproj
→ Next: restore the exact packages, then the publishing pipeline and your first local render

Renv: pinning package versions

Why pinning package versions matters



Same code + different package versions = different (or broken) results.

> "It works on my laptop but not yours."
> "It worked last year but not today."

renv = a locked, per-project package library

```
renv.lock     records the exact version of every package
renv/         a private package library just for this project
.Rprofile     activates renv when the project opens

→ Everyone restores the SAME versions. No more version drift.
```

Step 7 — install the exact packages

```R
install.packages("renv")   # only if RStudio doesn't prompt you
renv::restore()            # installs every package at the pinned version
```

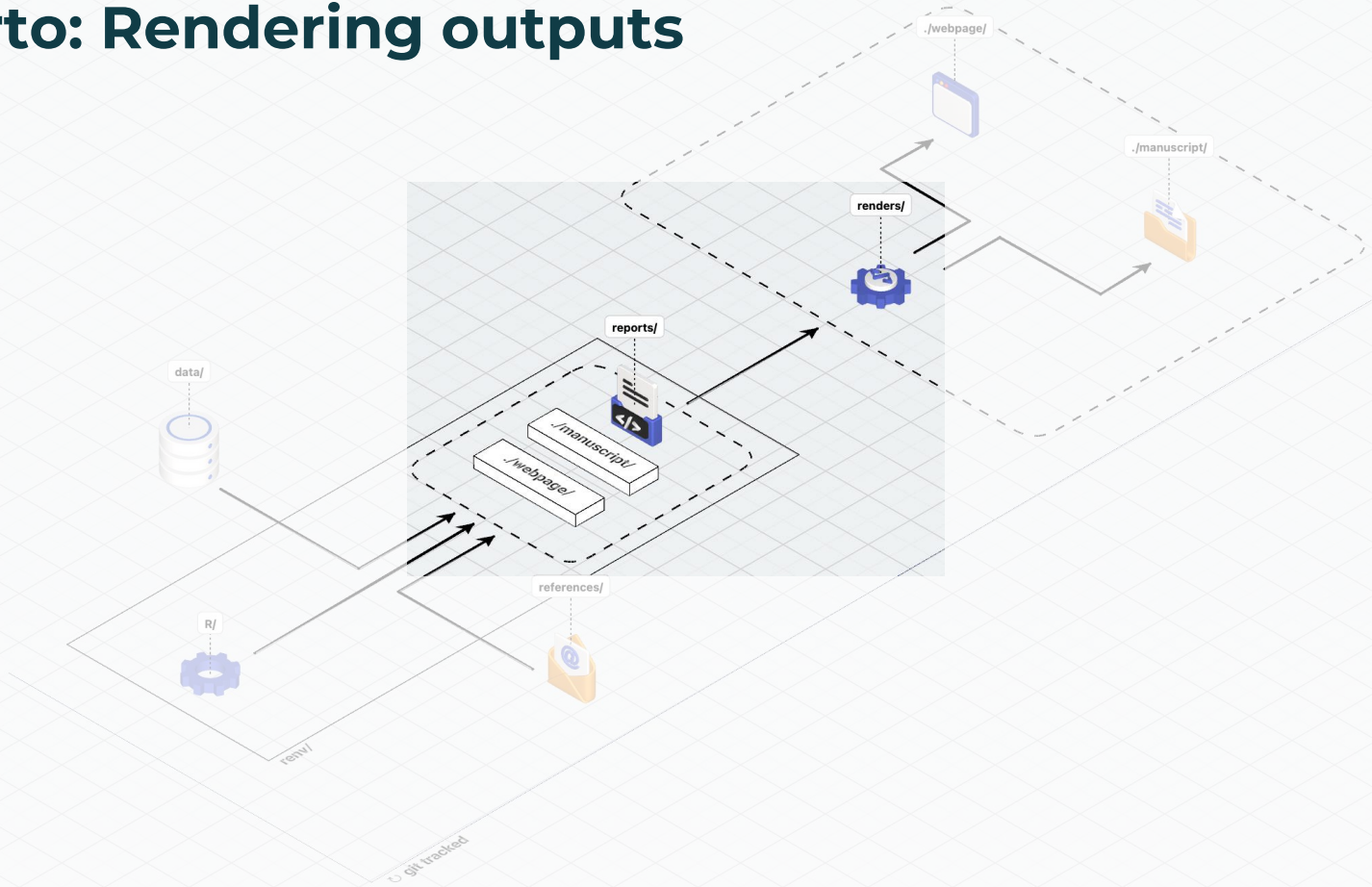
CHECK

renv reads renv.lock and installs what's missing.
This can take a few minutes — that's normal.
When it finishes, you have every package the project needs.

WATCH OUT

- If prompted "Do you want to proceed? [y/N]" — type y and Enter.
- First restore can compile packages from source. Mac users may be asked to install the Xcode Command Line Tools — accept it.
- It can take 5–15 min on a fresh machine. Let it finish; don't interrupt.
- Needs a stable internet connection.

Quarto: Rendering outputs

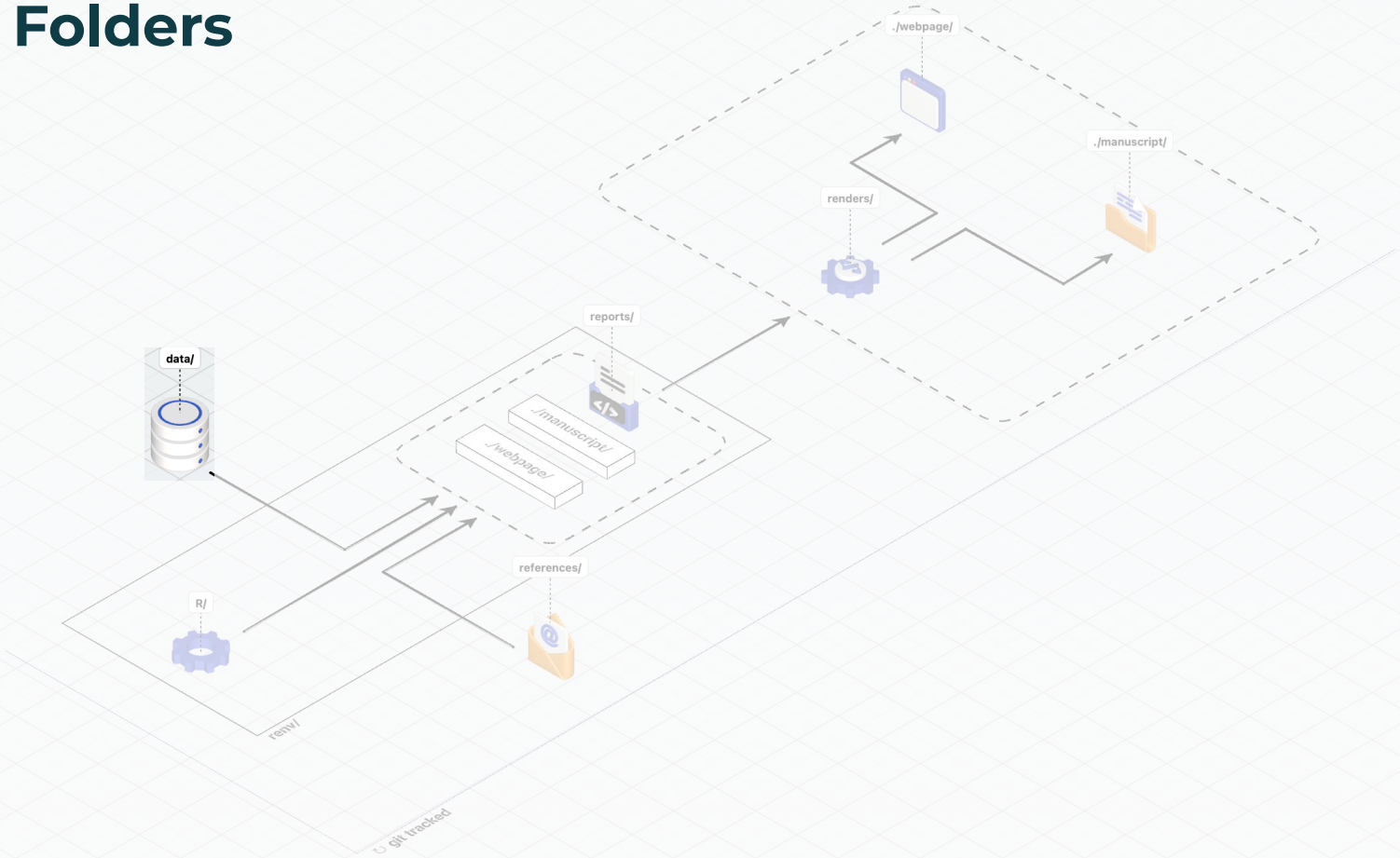


A report is a recipe, read top to bottom

```
reports/webpage/
  01-data-preparation.qmd   load + clean the data        → writes the processed cache
  02-descriptives.qmd       summarise it                 → uses the processed data
  03-analysis.qmd           model it                     → uses the processed data

Each page runs top → bottom. Setup first, results last.
They all call the shared functions in  R/functions/  (the kitchen techniques).
```

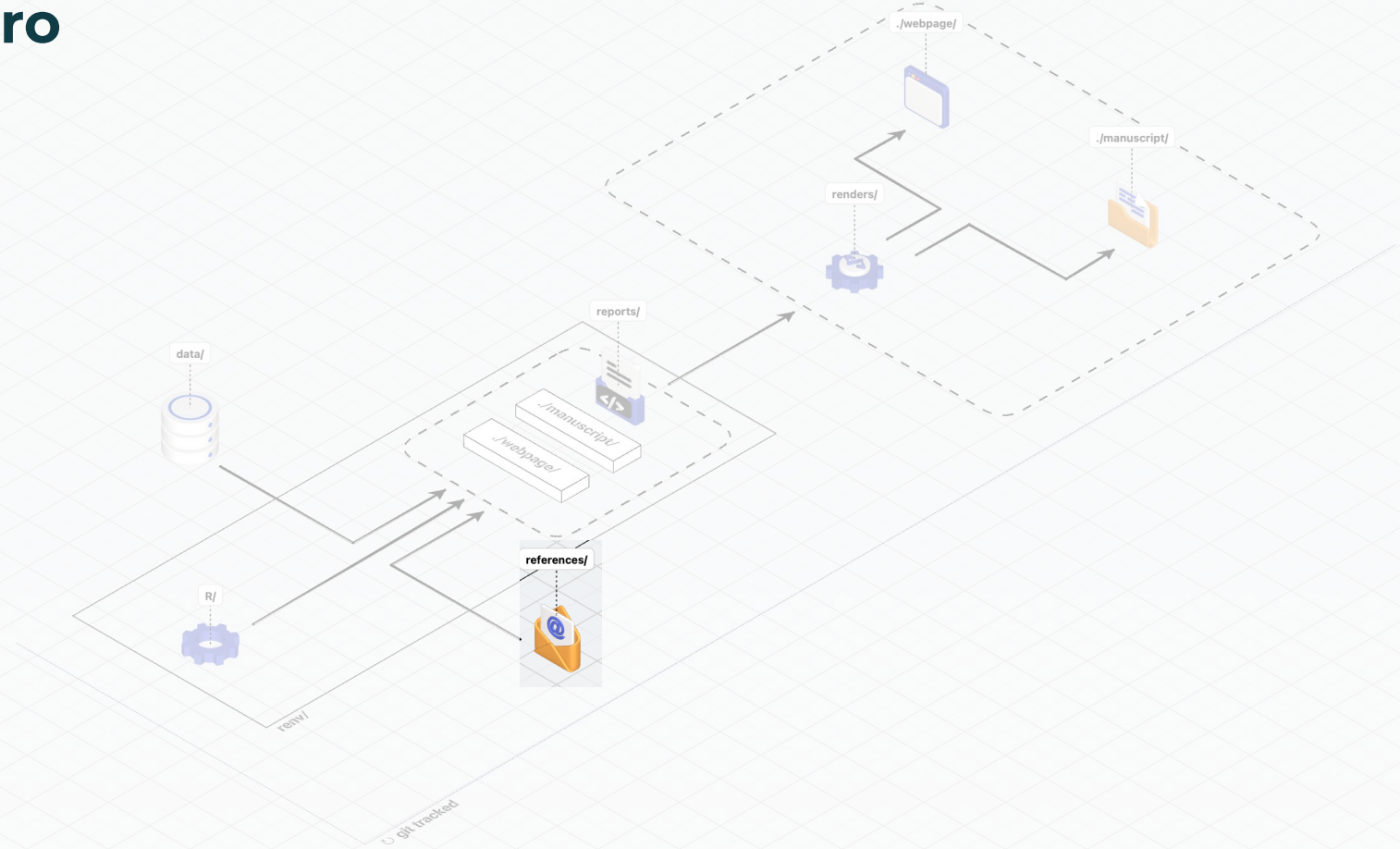
Data Folders



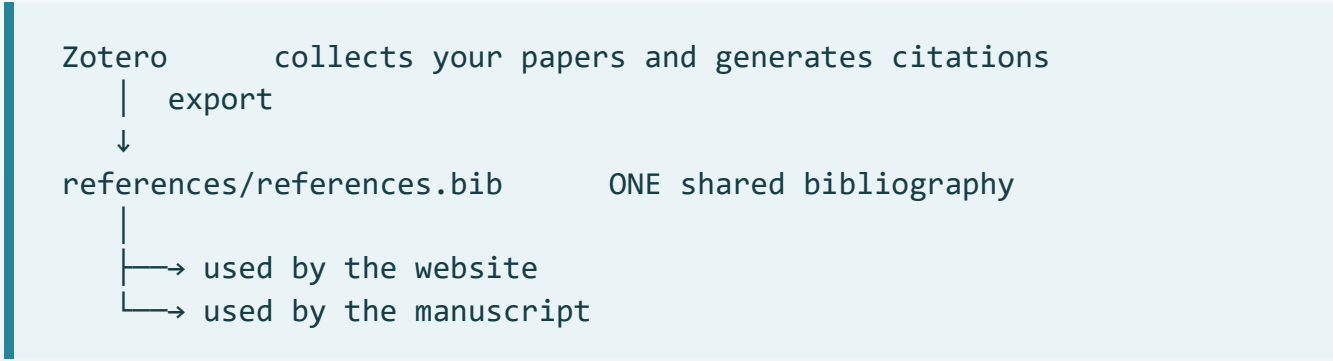
raw → processed → mock (what 01-data-preparation handles)

```
data/raw/        original data, exactly as received — never hand-edited (private)
data/processed/  cleaned, analysis-ready — created BY a script, can be rebuilt (private)
data/mock/       synthetic stand-in — committed, safe to share, used in the workshop
```

Zotero


./webpage/
./manuscript/
renders/
reports/
data/
./manuscript/
./webpage/
references/
R/
renv/
git tracked

Manage references once, cite everywhere



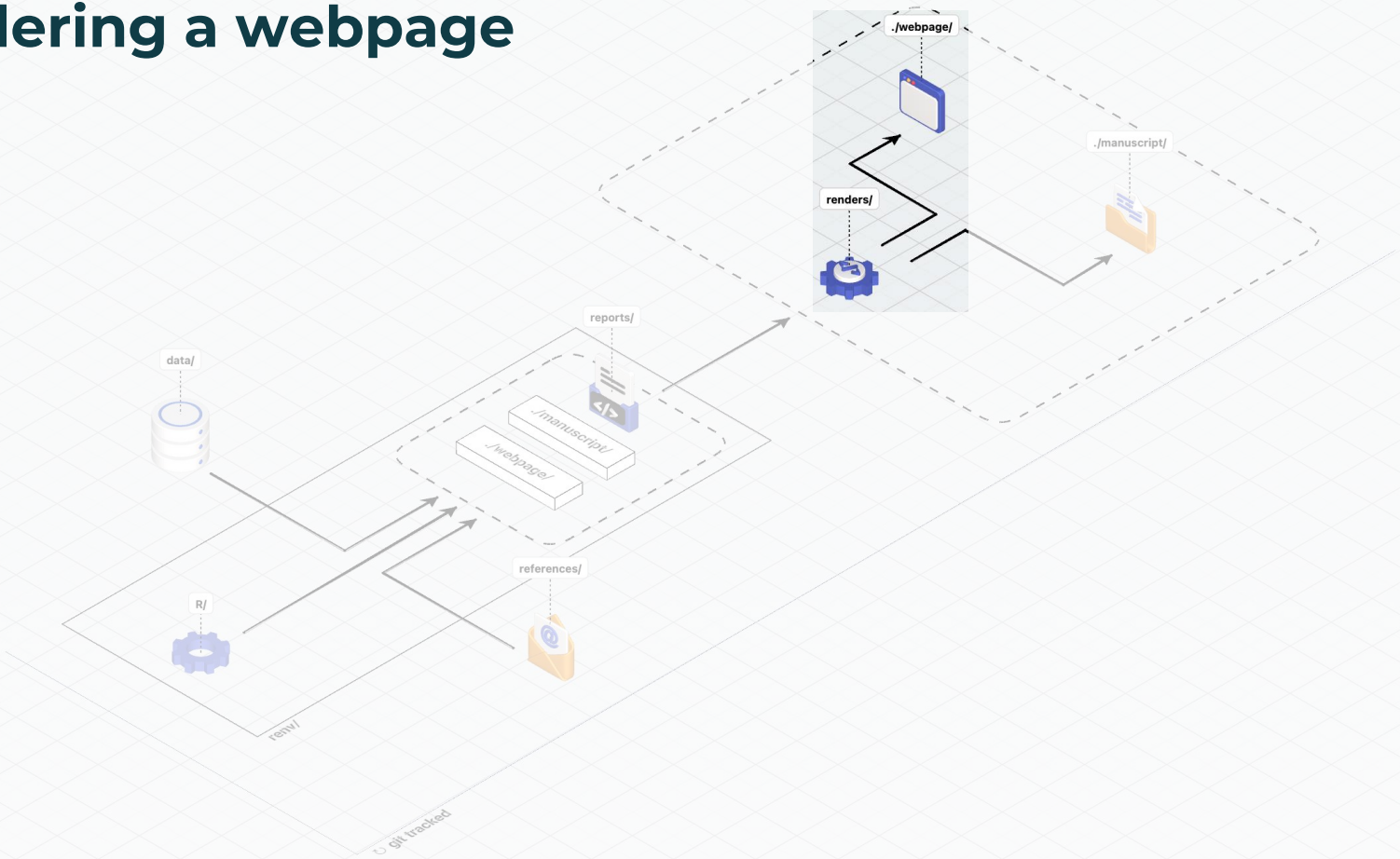
Why Zotero? Free and open (vs paid managers), and it plugs into Word, Quarto, and Overleaf. One library, cited everywhere.

Website and manuscript, same numbers

```
reports/webpage/      ──quarto render──→  renders/webpage/      (HTML website)
reports/manuscript/   ──quarto render──→  renders/manuscript/   (LaTeX / Word paper)
```

Both reuse R/. Same data, same functions, two presentations.

Rendering a webpage



Step 8 — build the site locally

BASH

```
quarto render reports/webpage
```

```
→ rebuilds renders/webpage/
Open renders/webpage/index.html to see it.
(Quarto ships inside RStudio — no separate install needed.)
```

WATCH OUT

- Run this in the TERMINAL tab, not the R Console.
- Render the WHOLE site (reports/webpage), not a single page — the first page builds the data/processed cache the others depend on.
- If "quarto: command not found", update RStudio (recent versions bundle Quarto) or install from quarto.org.

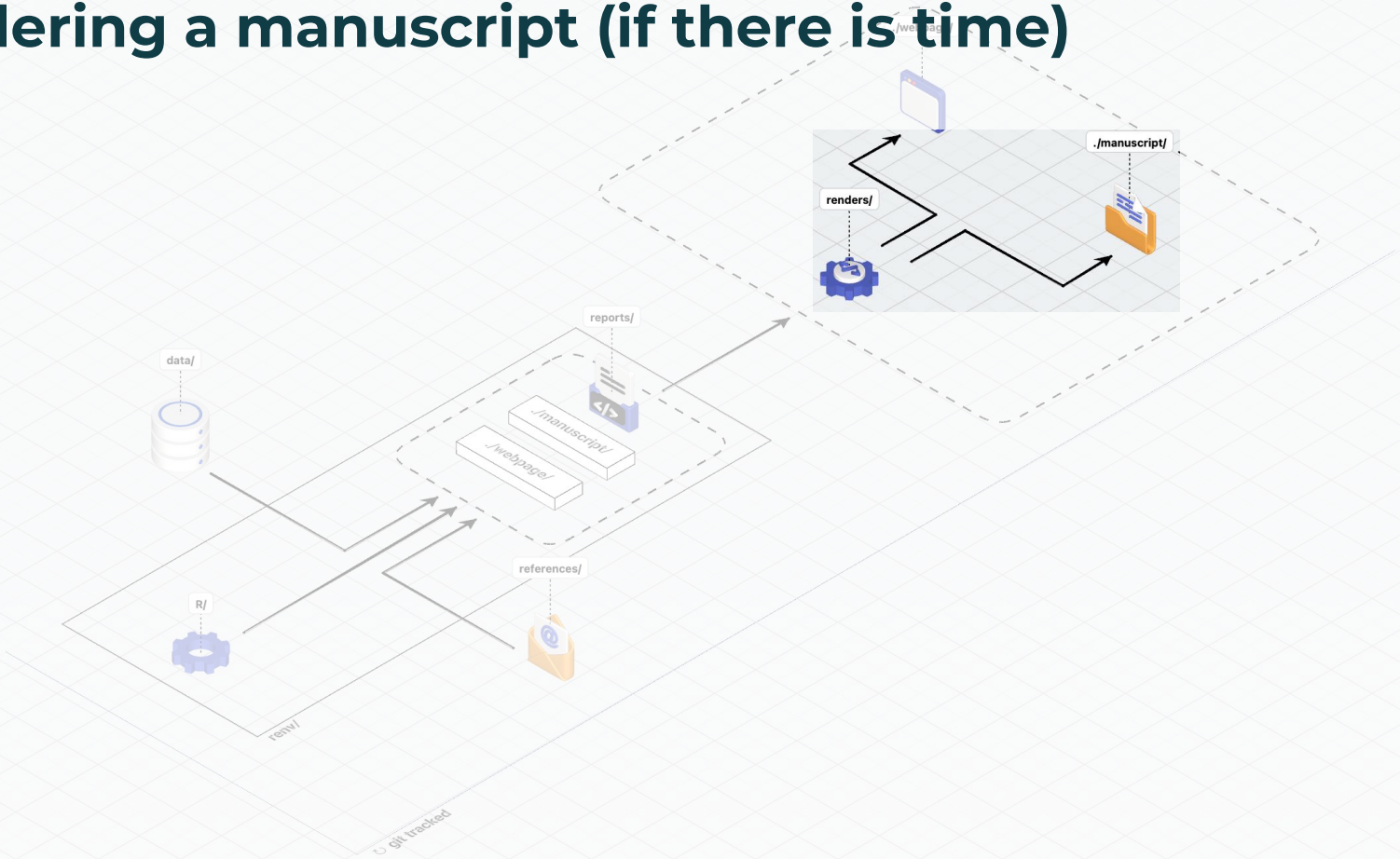
BREAK

Coffee break — 15:00–15:30

```
Back at 15:30.

✓ So far: environment restored · website rendered locally —
the Day 1 finish line
→ Next: the optional second output (the PDF manuscript), common
blockers, and
        the end-of-day checklist
```

Rendering a manuscript (if there is time)



The second output — the paper, and how co-authors join in

```
The website is today's finish line. The manuscript is the SAME engine, second dish.

To build the PDF you need a LaTeX engine (one-time install):
  in the R Console:
    install.packages("tinytex")
    tinytex::install_tinytex()

Then:
  quarto render reports/manuscript
    → generates the data-driven parts (methods, results, figures, tables)
    → renders/manuscript/main.tex assembles them into manuscript.pdf

Overleaf:
  the self-contained renders/manuscript/ folder can sync to Overleaf
  → online, collaborative LaTeX writing with co-authors
```

WATCH OUT

- tinytex downloads a full LaTeX distribution — needs internet and a few minutes.
- The manuscript needs the LaTeX engine above; the WEBSITE never does.

The day's usual suspects (and the fix)

```
"Git option greyed out"          → install Git (git-scm.com), restart RStudio
"Authentication failed" on push→ use a Personal Access Token, not a password
"Could not find function"        → package not loaded → did renv::restore() finish?
"cannot open file ... raw/..." → you're in real mode; default is mock (do nothing)
Website shows 404                → wait for the build; URL ends /renders/webpage/
"quarto: command not found"      → update RStudio / install from quarto.org
A single page won't render       → render the whole reports/webpage folder
```

Before you leave today

- [x] GitHub account created
- [x] Repository forked to your account
- [x] GitHub Pages on → your website loads
- [x] First commit made (README link points to your site)
- [x] R and RStudio installed
- [x] Fork cloned and opened via the .Rproj
- [x] renv::restore() finished successfully
- [x] quarto render reports/webpage works locally

Day 2 — analyse the data in R

```
tomorrow we:
  - work inside R/ and the report pages
  - import, clean, and inspect the data
  - build descriptives, figures, and models
  - use branches and merges to collaborate in pairs
```

See you tomorrow

We're happily staying for extra questions

info@thedataflowcompany.com
www.thedataflowcompany.com